

高。而且，很多系统的业务逻辑部分并没有提供 API，这样，集成难度就会更大。

#### 4. 业务流程集成

业务流程集成也称为过程集成，这种集成超越了数据和系统，它由一系列基于标准的、统一数据格式的工作流组成。当进行业务流程集成时，企业必须对各种业务信息的交换进行定义、授权和管理，以便改进操作、减少成本、提高响应速度。

业务流程集成不仅要提供底层应用支撑系统之间的互连，同时要实现存在于企业内部的应用之间、本企业和其他合作伙伴之间的端到端的业务流程的管理，它包括应用集成、B2B 集成、自动化业务流程管理、人工流程管理、企业门户，以及对所有应用系统和流程的管理和监控等。

#### 5. 企业间应用集成

EAI 技术可以适用于大多数要实施电子商务的企业，以及企业之间的应用集成。EAI 使得应用集成架构里的客户和业务伙伴都可以通过集成供应链内的所有应用和数据库实现信息共享。也就是说，能够使企业充分利用外部资源。例如，一些企业的 SCM 系统可能包括交易系统，EAI 技术可以首先在交易双方之间创建连接，然后再共享数据和业务过程；企业要顺利开展电子商务，可以利用 EAI 技术，使企业的信息系统与合作伙伴的信息系统之间能够实现无缝而及时的通信。

### 6.8.2 事件驱动的企业应用集成

EAI 提供了一个开放的框架，使现有的应用系统和数据库可根据企业业务的需要实现集成，并且能快速开发新的应用系统，使企业既可以保护已有的投资，又可以根据市场和业务的需求重新整合原有的信息系统，产生新的竞争力。

事件技术是一种非常适合用于分布式异构系统之间松散耦合的协作技术，基于事件驱动的 EAI 系统同样继承了这一优点。

#### 1. 事件驱动架构

事件驱动架构（Event-Driven Architecture，EDA）是一种设计和构建应用的方法，其中事件触发消息在独立的、非耦合的模块之间（它们之间不需要知道对方）传递。事件源通常发送消息到中间件或消息代理，订阅者订阅这个消息。因为事件消息用发布/订阅方式通过消息代理传输，一个事件可以传送给多个订阅者。EDA 的主要优势在于，它允许企业通过事件管理来标识和响应一个或多个系统中的事件。这些事件通过 EDA 被收集起来，可以被分析和定义相关的模式，并可以构建信息模型来解决问题。EDA 的主要特点如下：

（1）异步。EDA 主要支持异步活动，消息在发出后，不必再关心是否能收到响应，同样也不必在源和目的系统之间维持一条活的链路。

（2）发布/订阅。EDA 主要支持多对多的交互。在 EDA 中，系统发布一个关于事件的消息到网络中，多个其他的已经订阅和授权的系统就可以收到消息并做出响应。

（3）解耦。EDA 允许消息的发布者不知道订阅者是谁，反之亦然。也就是说，消息在两个系统间交互时，根本不需要知道对方的详细信息。

支持事件和消息技术的主要模块包括以下两个：

(1) 异步消息机制。EDA 必须要保证当事件发生时，相应的系统能发送异步消息。既然异步消息不需要立即从接收者处收到响应，也不需要保证消息的传输，那么就要考虑到事件的发生和处理会暂时不可用。

(2) 事件管理。EDA 必须保证有一个系统用来识别、定义和聚集事件，这样，事件就可以像企业数据和业务流程那样被统一管理。

## 2. 事件驱动的 EAI

在企业经营过程中，所有业务都是事件驱动的。在 EDA 系统中，事件产生者发布事件，事件消费者接收事件，所以 EDA 极大地改善了企业对各种看似无关的事件的响应能力，而这些事件往往会对企业造成影响。通过提供即时过滤、聚集和关联事件的功能，EDA 能够以极快的速度检测有可能对企业造成威胁或为企业提供商业机遇的事件和模式，并且为企业提供对此做出即时反应的能力。事件驱动的 EAI 框架如图 6-15 所示。

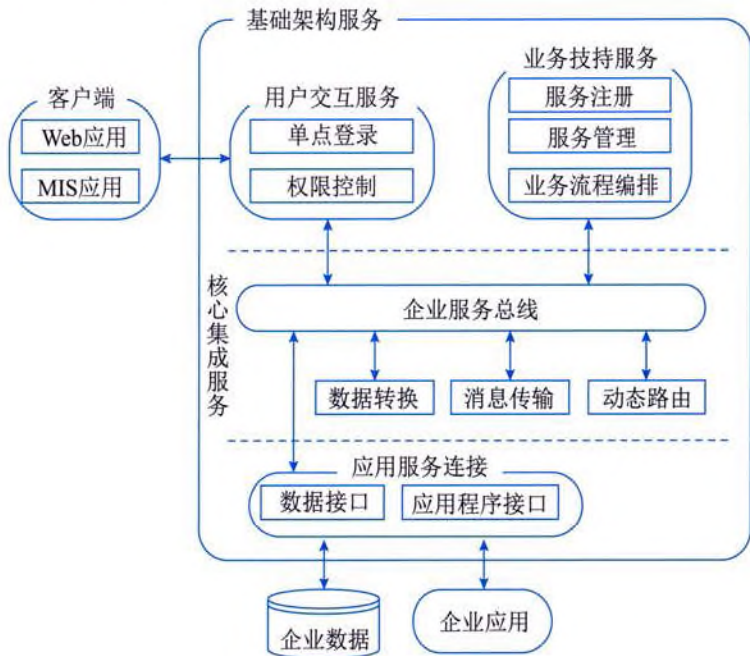


图 6-15 事件驱动的 EAI 框架

图 6-15 中的各个功能实体都以服务的形式出现，是在特定层次上为特定应用提供服务的基础设施。整个架构中的服务由以下 5 层构成：

(1) 企业服务总线（Enterprise Service Bus, ESB）。这是面向服务体系中的基础架构，各个服务通过总线来互相访问。

(2) 应用服务层。主要是指需要集成的各个应用系统和数据库。

(3) 总线接入层。提供适配器服务，支持多种主流应用的接入协议，使用户可以访问各个应用服务，并通过消息机制使各种应用接入 ESB，使用 ESB 的各种服务。



(4) 核心服务层。提供多种 ESB 所需的必要服务支持，例如，消息分发 / 订阅、队列、目录服务和数据转换 / 映射服务等。

(5) 业务支持层。侧重在业务支持上，通过通用、标准的对象和服务模型，可以在这一层上定义可复用的和基于企业标准的业务流程。同时，还提供统一的用户交互服务。建立在 ESB 上的用户交互服务可以很小巧，并关注于各自交互的特点。

事件驱动的 EAI 框架基于面向服务技术，通过各类适配器服务接口将企业应用封装成统一的应用服务，然后发布到目录服务中心，并通过 ESB 中的基础核心服务（例如，统一数据格式和消息传递等）来实现各个应用系统之间的通信交互。在该框架中，应用服务既可以是已有的应用，也可以是新开发的应用。任何应用都以独立服务的形式连接到系统中，方式灵活，简单快速，真正实现了“即插即用”。

当在事件驱动的 EAI 框架下需要进行过程集成和业务集成时，首先通过业务流程定义服务，并根据事件驱动的模式将已经注册的应用服务在一定的规则下组成相应的业务流程链。业务集成模型的实现是由集成引擎调用应用服务的接口实现数据的存取，并通过消息引擎在各个应用服务间传递路由数据，实现定义的业务流程。

## 第7章 软件工程

软件工程是指应用计算机科学、数学及管理科学等原理，以工程化的原则和方法来解决软件问题的工程，其目的是提高软件生产率、提高软件质量、降低软件成本。IEEE 对软件工程的定义是：将系统的、规范的、可度量的工程化方法应用于软件开发、运行和维护的全过程及上述方法的研究。

软件工程由方法、工具和过程三个部分组成。软件工程方法是完成软件工程项目的手段，它支持整个软件生命周期；软件工程使用的工具是人们在开发软件的活动中智力和体力的扩展与延伸，它自动或半自动地支持软件的开发和管理，支持各种软件文档的生成；软件工程中的过程贯穿软件开发的各个环节，管理人员在软件工程过程中，要对软件开发的质量、进度、成本进行评估、管理和控制，包括人员组织、计划跟踪与控制、成本估算、质量保证和配置管理等。

### 7.1 软件生命周期

软件产品从形成概念开始，经过开发、使用和维护，直到最后退役的全过程称为软件生命周期或生存周期。一个完整的软件生命周期是以需求为出发点，从提出软件开发计划的那一刻开始，直到软件在实际应用中完全报废为止。软件生命周期的提出是为了更好地管理、维护和升级软件，其中更大的意义在于管理软件开发的步骤和方法。

目前，划分软件生命周期阶段的方法有许多种，软件规模、种类、开发方式和开发环境，以及开发时使用的方法论都影响软件生命周期阶段的划分。对软件生命周期各阶段进行划分，必须遵循一条基本的原则，那就是各阶段的任务应尽可能地相对独立，同一阶段各项任务的性质应尽可能地相同，从而降低每个阶段任务的复杂度，减少不同阶段任务之间的联系，有利于软件工程的组织和管理。

#### 1. 软件生命周期过程

在国家标准《系统与软件工程 软件生存周期过程（GB/T 8566—2022）》中，将软件生存周期中可能执行的活动分为5个基本过程、9个支持过程和7个组织过程。每个生存周期过程划分为一组活动，每一项活动进一步划分为一组任务。

（1）基本过程。基本过程供各主要参与方在软件生存周期期间使用，主要参与方是发起或完成软件产品开发、运行或维护的组织。基本过程分为获取过程、供应过程、开发过程、运作过程和维护过程。获取过程是指为获取系统、软件产品或软件服务的组织（即需方）而定义的活动；供应过程是指为向需方提供系统、软件产品或软件服务的组织（即供方）而定义的活动；开发过程是指为定义并开发软件产品的组织（即开发方）而定义的活动，包括需求分析、设计编码、集成、测试以及与软件产品有关的安装和验收等活动；运作过程是指为在规定的环境中为其用户提供运行计算机系统服务的组织（即操作方）而定义的活动；维护过程是指为提供维



护软件产品服务的组织（即维护方）而定义的活动，也就是对软件的修改进行管理，使它保持合适的运行状态，包括软件产品的迁移和退役。

（2）支持过程。支持过程作为一个有机组成部分支持其他过程，以便取得软件项目的成功，并提高软件项目的质量。支持过程包括文档编制过程、配置管理过程、质量保证过程、验证过程、确认过程、联合评审过程、审核过程、问题解决过程和易用性过程。根据需要，支持过程可被其他过程应用和执行。文档编制过程是指为记录生存周期过程所产生的信息而定义的活动；配置管理过程是指定义配置管理活动；质量保证过程是指为客观地保证软件产品和过程符合规定的需求以及已建立的计划而定义的活动，联合评审、审核、验证和确认可以作为质量保证技术使用；验证过程是指根据软件项目需求，按不同深度为需方、供方或某独立方验证软件产品而定义的活动；确认过程是指为需方、供方或某独立方确认软件项目的软件产品而定义的活动；联合评审过程是指为评价一项活动的状态和产品而定义的活动，该过程可由任何两方应用，其中一方（评审方）以联合讨论会的形式评审另一方（被评审方）；审核过程是指为判定符合需求、计划和合同而定义的活动，该过程可由任何两方应用，其中一方（评审方）审核另一方（被评审方）的软件产品或活动；问题解决过程是指为分析和解决问题（包括不合格）而定义的活动，不论问题的性质或来源如何，它们都是在实施开发、运作、维护或其他过程期间暴露出来的；易用性过程是指为易用性专业人员而定义的活动。

（3）组织过程。组织过程可被某个组织用来建立和实现由相关的生存周期过程和人员组成的基础结构，并不断改进这种结构和过程。应用这些过程通常超出特定的项目和合同的范围，但是，这些特定项目和合同的经验教训有助于改善组织状况。组织过程包括管理过程、基础设施过程、改进过程、人力资源过程、资产管理过程、重用大纲管理过程和领域工程过程。管理过程是指为生存周期中的管理（包括项目管理）而定义的基本活动；基础设施过程是指为建立生存周期过程基础结构而定义的基本活动；改进过程是指为某一组织建立、测量、控制和改进其生存周期过程而定义的需要执行的基本活动；人力资源过程是指为给组织或项目提供拥有技能和知识的员工而定义的活动；资产管理过程是指为组织的资产管理人员而定义的活动；重用大纲管理过程是指为组织的复用大纲主管而定义的活动；领域工程过程是指为领域模型、领域架构的确定及该领域资产的开发和维护而定义的活动。

## 2. 软件生命周期各阶段的任务

根据国家标准 GB/T 8566—2022，软件生命周期可以划分为可行性研究、需求分析、概要设计、详细设计、实现、组装测试、确认测试、使用、维护、退役 10 个阶段，各自分别对应于软件生存周期的基本过程，如图 7-1 所示。

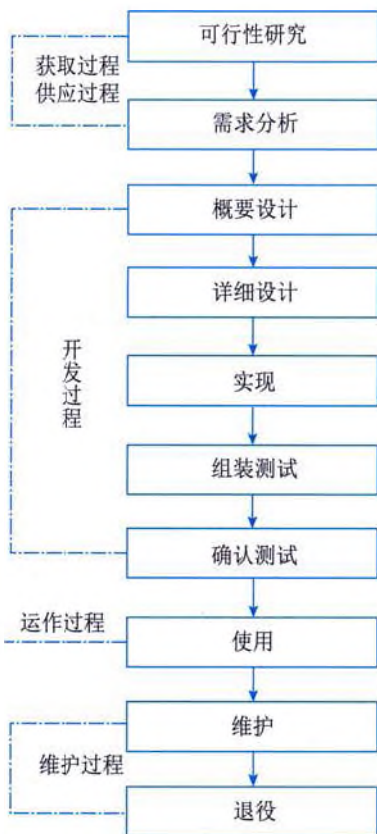


图 7-1 软件生命周期与 5 个基本过程的对应关系



(1) 可行性研究和项目开发计划。通过分析用户提出的软件开发要求，确定软件项目的性质、目标和规模，得出可行性研究报告。如果可行性研究的结果是可行的，就要制定详细的项目开发计划。这两个活动通常被整合在一起进行，在实际工作中通常把它们归类到同一个阶段中。

(2) 需求分析。需求分析工作是软件生命周期中重要的一步，也是决定性的一步。只有通过需求分析，才能把软件功能和性能的总体概念描述为具体的软件需求规格说明，从而奠定软件开发的基础。

(3) 概要设计。根据软件需求规格说明建立软件系统的总体结构和模块间的关系，定义各功能模块接口，设计全局数据库或数据结构，规定设计约束，制定组装测试计划。

(4) 详细设计。将各模块要实现的功能用相应的设计工具详细描述出来。

(5) 实现。写出正确的、易理解的和易维护的程序模块。程序员根据详细设计文档将详细设计转化为程序，完成单元测试。

(6) 组装测试（集成测试）。将经过单元测试的模块逐步进行组装和测试。

(7) 确认测试。测试系统是否达到了系统需求，按照规格说明书的规定，由用户（或在用户积极参与下）对系统进行验收。必要时，还可以再通过现场测试或并行运行等方法对系统进行进一步的测试。

(8) 使用。将软件安装在用户确定的运行环境中，测试通过后移交用户使用。在软件的使用过程中，客户和维护人员必须认真收集发现的软件错误，定期或阶段性地撰写软件问题报告和软件修改报告。

(9) 维护。通过各种必要的维护活动使系统持久地满足用户的需要。

(10) 退役。终止对软件产品的支持，软件停止使用。

## 7.2 软件开发方法与模型

软件开发方法是指软件开发过程所遵循的办法和步骤，从不同的角度可以对软件开发方法进行不同的分类。从开发风范上看，可分为自顶向下的开发方法与自底向上的开发方法。在实际软件开发中，经常是两种方法结合使用，只不过是应用于开发的不同阶段和以何者为主而已。

软件开发模型则给出了软件开发活动各阶段之间的关系，它是软件开发过程的概括，是软件工程的重要内容。软件开发模型为软件工程管理提供了里程碑和进度表，为软件开发过程提供了原则和方法。

### 7.2.1 传统软件开发模型

软件开发模型大体上可分为三种类型：第一种是以软件需求完全确定为前提的瀑布模型；第二种是在软件开发初始阶段只能提供基本需求时采用的迭代式或渐进式开发模型，例如，喷泉模型、螺旋模型、统一开发过程和敏捷方法等；第三种是以形式化开发方法为基础的变换模型。

## 1. 瀑布模型

软件开发生命周期（Software Development Life Cycle, SDLC）模型的发展体现了软件理论的发展。在最早的时候，软件的开发过程处于无序和混乱的状况，人们为了能够控制软件的开发过程，就将软件开发严格区分为多个不同的阶段，并在阶段之间加上严格的审查，这就是瀑布模型产生的起因。

瀑布模型是一种严格定义方法，它将软件开发的过程分为软件计划、需求分析、软件设计、程序编码、软件测试和运行维护 6 个阶段，形如瀑布流水，最终得到软件产品，如图 7-2 所示。

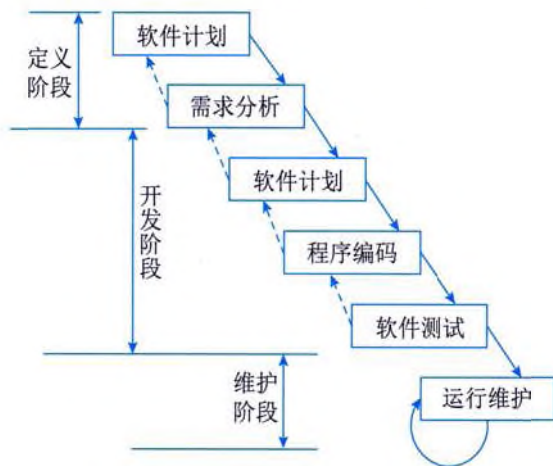


图 7-2 瀑布模型

瀑布模型是一个线性顺序模型，支持线性开发。它假设当线性序列完成之后就能交付一个完善的系统，并没有考虑软件的演化特征。其优点是强调开发的阶段性、早期计划和需求调查以及产品测试，以这样严格的方式构造软件，开发人员很清楚每一步应该做什么，有利于项目管理。

然而，在瀑布模型中，依赖于早期进行的需求调查，不能适应需求的变化；由于是单一流程，开发中的经验教训不能反馈应用于本产品的过程；风险往往推迟至后期的开发阶段才显露出来，从而失去了及早纠正的机会。在瀑布模型中，需求或设计中的错误往往只有到了项目后期才能够被发现，对于项目风险的控制能力较弱，从而导致项目常常延期完成，开发费用超出预算。

## 2. 演化模型

演化模型主要针对事先不能完整定义需求的软件开发，是在快速开发一个原型的基础上，根据用户在调用原型的过程中提出的反馈意见和建议，对原型进行改进，获得原型的新版本，重复这一过程，直到演化成最终的软件产品。

演化模型的主要优点是，任何功能一经开发就能进入测试，以便验证是否符合产品需求，可以帮助引导出高质量的产品要求。其主要缺点是，如果不加控制地让用户接触开发中尚未稳定的功能，可能对开发人员及用户都会产生负面的影响。



### 3. 螺旋模型

螺旋模型是瀑布模型与演化模型相结合，并加入两者所忽略的风险分析所建立的一种软件开发模型。螺旋模型是一种演化软件过程模型，它将原型实现的迭代特征与线性顺序模型中的控制和系统化的方面结合起来，使软件的增量版本的快速开发成为可能。在螺旋模型中，软件开发是一系列的增量发布。

螺旋模型沿着螺旋线进行若干次迭代，每次迭代都包括制订计划、风险分析、实施工程和客户评估4个方面的工作。螺旋模型强调风险分析，使得开发人员和用户对每个演化层出现的风险有所了解，继而做出应有的反应。因此，它特别适用于庞大、复杂并具有高风险的系统。

与瀑布模型相比，螺旋模型支持用户需求的动态变化，为用户参与软件开发的所有关键决策提供了方便，有助于提高软件的适应能力，并且为项目管理人员及时调整管理决策提供了便利，从而降低了软件开发的开发风险。在使用螺旋模型进行软件开发时，需要开发人员具有相当丰富的风险评估经验和专门知识。另外，过多的迭代次数会增加开发成本，延迟提交时间。

### 4. 喷泉模型

喷泉模型是一种以用户需求为动力，以对象为驱动力的模型，主要用于描述面向对象的软件开发过程。该模型认为软件开发过程自下而上的各阶段是相互重叠和多次反复的，就像水喷上去又可以落下来，类似一个喷泉。各个开发阶段没有特定的次序要求，并且可以交互进行，可以在某个开发阶段随时补充其他任何开发阶段中的遗漏。

在喷泉模型中，各活动之间无明显边界，例如，分析和设计之间没有明显的边界。这种特性称为无间隙性。由于对象概念的引入，只用类和关系来表达分析、设计和实现等活动，从而可以较容易地实现活动的迭代和无间隙，提高软件项目开发效率，节省开发时间。

### 5. 变换模型

变换模型是基于形式化规格说明语言和程序变换的软件开发模型，它对形式化的软件规格说明进行一系列自动或半自动的程序变换，最后映射为计算机能够接受的软件系统。为了确认形式化规格说明与软件需求的一致性，往往以形式化规格说明为基础开发一个软件原型，用户可以从人机界面、系统主要功能和性能等方面对原型进行评审。必要时，可以修改软件需求、形式化规格说明和原型，直至原型被确认为止。这时，开发人员即可对形式化的规格说明进行一系列的程序变换，直至生成计算机可以接受的目标代码。

变换模型的优点是解决了代码结构经多次修改而变坏的问题，减少了许多中间步骤（例如，设计、编码和测试等）。但是，变换模型仍有较大的局限性，以形式化开发方法为基础的变换模型需要严格的数学理论和一整套开发环境的支持。

### 6. 智能模型

智能模型也称为基于知识的软件开发模型，它综合了上述若干模型，并把专家系统结合在一起。该模型应用基于规则的系统，采用规约和推理机制，帮助开发人员完成开发工作，并使维护在系统规格说明一级进行。为此，需要建立知识库，将模型本身、软件工程知识与特定领域的知识分别存入知识库。